ESCROW SYSTEM FOR FREELANCERS

A Beginner-Friendly Complete Project Walkthrough & Guide

& Ultimate Interview Preparation Blueprint

Created for: Harini Sreeja

Technology Stack: HTML5, CSS3, JavaScript, Firebase, PHP

Date: July 2026

Table of Contents

1. Introduction to Web Development (For Absolute Beginners)

Understanding HTML, CSS, JavaScript, PHP, and Firebase through real-world analogies.

2. Project Architecture & Workflow

How pages link together and how user authentication, database writes, and search flows work.

3. Pin-to-Pin Code Explanation

Understanding the core Javascript files, Firebase configs, and PHP script line by line.

4. Demystifying the "Escrow Payment" Aspect

Explaining the mockup nature of the escrow part and how to handle this in interviews.

5. Comprehensive Interview Q&A Prep

15+ questions and strategic answers to ace your viva/interview like a pro.

1. Introduction to Web Development (For Absolute Beginners)

Before exploring the code, let us understand the basic terms used to build this website. If you are new to web development, think of this project like building a physical house:

- **HTML (HyperText Markup Language):** The structural skeleton of the house. It defines where the walls, doors, windows, and roof go. In our website, files like index.html and contact.html define where text, headers, images, forms, and buttons are located.
- **CSS (Cascading Style Sheets):** The interior design and paint. It styles the skeleton, deciding the colors of the walls, fonts, margins, alignments, and background images. Our file css styles.css makes the layout clean and modern.
- **JavaScript (JS):** The electricity and plumbing. It makes the house functional. When a user clicks a button, JS runs in the background to handle the action. It handles user login, signs users out, filters jobs, and runs search queries.
- **Firebase:** A secure cloud server dashboard. Instead of building a complex database from scratch, we use Google Firebase. It does two things in this project:
  1. Firebase Authentication: Validates and secures user accounts (sign up/sign in).
  2. Cloud Firestore: A cloud database that saves user details (first name, last name, email) in a "users" collection.

- PHP (Hypertext Preprocessor): A server-side script. In our project, send_email.php acts like a postman. It runs on the web hosting server to format contact messages and email them directly to your Gmail account.

2. Project Architecture & Workflow

This application is structured as a multi-page web application. Here is the step-by-step user journey and technical workflow:

Step 1: User Lands on Homepage (index.html)

The homepage shows a hero banner, key features, demanding jobs list, brand trust icons, top-rated freelancers, and a footer. It has navigation links to other pages. The header contains a Login button.

Step 2: Sign Up (Registration) Flow

1. User clicks "Sign Up" or is redirected to Sign up.html.
2. The user fills in First Name, Last Name, Email, Password, and Confirm Password.
3. Clicking "Sign Up" triggers the script firebaseauth.js.
4. The script checks if the password matches the confirm password. If it does, it calls Firebase Authentication to create a user account.
5. Once the account is created, the script writes a document in the "users" collection of Cloud Firestore under the user's unique identifier (UID). This stores their names and email securely.
6. The user is notified and redirected.

Step 3: Sign In (Login) Flow

1. User visits Sign in.html.
2. User submits their email and password.
3. The script firebaseSignin.js passes these credentials to Firebase Authentication for validation.
4. If validation succeeds, Firebase returns a unique User ID (UID).
5. The script saves this UID inside the browser's LocalStorage under the key "loggedInUserId" (this keeps the session active even if the user refreshes the page).
6. The browser redirects the user to index.html.

Step 4: Active Session Checking & Personalization

When the user is on the homepage, Logout.js executes in the background:
1. It checks if there is a "loggedInUserId" stored in the LocalStorage.
2. If found, it fetches the user's profile details (First Name, Last Name, Email) from Firestore database using that ID.
3. It dynamically updates the HTML page to display the user's name and email in the header or profile section.
4. If the user clicks the "Logout" button, the script deletes "loggedInUserId" from LocalStorage, signs out of Firebase, and redirects to Sign in.html.

Step 5: Job Exploration & Search (jobs/job.html)

1. Users can search jobs by title. Typing in the search input triggers jobSearch.js. It loops through all jobs on the page and hides those whose titles do not match the input.
2. Users can filter jobs by type (Full Time, Part Time, Freelancer). Clicking a filter category triggers job-list.js. It hides non-matching jobs by applying a CSS class ".delete" which has display:none in CSS.

Step 6: Contact Support (contact.html)

1. The user fills out a contact form (Name, Email, Message).
2. Submitting the form sends a POST request to send_email.php on the backend server.
3. The PHP script processes the input and emails it to satyavarapuharinisreeja@gmail.com.

3. Pin-to-Pin Code Explanation

Let us break down the core logic of each JavaScript and PHP file. This is crucial for answering code-specific questions.

A. User Registration Logic (firebaseauth.js)

This file handles form submission on Sign up.html. Here are the key code actions:

- **Firebase Config:** Line 10 defines the firebaseConfig object containing apiKey, authDomain, and projectId. These are credentials that connect your frontend web pages to your Firebase backend project.
- **Submit Listener:** Line 33 attaches an event listener to the "Sign Up" button. event.preventDefault() stops the page from reloading, allowing JavaScript to handle the request.
- **Password Check:** Line 41 checks: if(password !== confirmPassword). If they do not match, it displays an error using the showMessage() helper and stops registration.
- **Create User:** Line 49 calls createUserWithEmailAndPassword(auth, email, password). Firebase registers the email and encrypts the password.
- **Save Profile:** Line 60 uses setDoc(docRef, userData) where docRef represents db -> users -> user.uid. It writes the first name, last name, and email to Cloud Firestore, enabling personalized profile retrieval.

B. User Login Logic (firebaseSignin.js)

This file handles Sign in.html. Key features:

- **Sign In Action:** Line 37 calls signInWithEmailAndPassword(auth, email, password) to verify credentials against Firebase Auth database.
- **Session Storage:** Line 41: localStorage.setItem("loggedInUserId", user.uid). This stores the user's unique ID in the browser's storage. Even if the user closes and opens the tab, they remain logged in.
- **Redirect:** Line 42 redirects the logged-in user to index.html.

C. Profile Loading & Logout (Logout.js)

This script manages session persistence and signing out:

- **AuthState Listener:** Line 20: onAuthStateChanged(auth, (user) => { ... }) runs whenever the auth state changes. It gets the loggedInUserId from LocalStorage.
- **Get Profile:** Line 24-25: doc(db, "users", loggedInUserId) and getDoc(docRef) retrieve the user's document. It dynamically injects their name and email into HTML elements: loggedUserFName, loggedUserLName, and loggedUserEmail.

- **Logout Action:** Line 47-52: Clears localStorage and calls signOut(auth) to log out of Firebase, then redirects to the Sign In page.

D. Job Filtering (jobs/job-list.js)

This script filters jobs dynamically without reloading the page:

- **Remove Active Class:** Line 6 loops through the filter buttons and removes the "active" CSS class, then adds it to the clicked button.
- **Toggle ".delete" Class:** Line 10-15 loops through all job list items. If a job's category (data-item) matches the selected category (data-target), or if "All" is selected, it shows the item. Otherwise, it adds the ".delete" class (which hides the item via CSS: display: none).

E. Job Searching (jobs/jobSearch.js)

Handles searching jobs by matching letters in real-time:

- **Input Listener:** Line 5: searchInput.addEventListener("input", ...) triggers on every keypress.
- **Text Check:** Line 8-9 extracts the text in the job's <h3> (e.g. "Web Developer"). It checks if the title includes the search letters (both converted to lowercase using .toLowerCase() to make it case-insensitive).
- **Show/Hide:** Line 10-12 sets item.style.display to "block" if it matches, and "none" to hide it if it doesn't.

F. Mobile Menu Drawer (toggle.js)

Toggles the side navigation drawer on mobile viewports. Clicking the hamburger icon (bar) toggles the ".active" class on the navigation list (menu), sliding it into view using CSS transitions.

G. Server-Side Email Dispatcher (send_email.php)

This script processes contact form submissions from contact.html on the server side:

- **POST Verification:** Line 3 checks if($_SERVER["REQUEST_METHOD"] == "POST"). This ensures only data submitted through the contact form is processed, blocking direct URL access.
- **Email Parameters:** Line 8-10 defines the recipient ($to = "satyavarapuharinisreeja@gmail.com"), the email subject, and custom headers (From, Reply-To, Content-Type) for professional formatting.
- **Mail function:** Line 15: mail($to, $subject, $message, $headers). This sends the message using the server's built-in mail transport agent (MTA). If it succeeds, it returns status code 200 (Success); otherwise, 500 (Server Error).

4. Demystifying the "Escrow Payment" Aspect

An Escrow system is a financial agreement where a trusted third party holds funds until the client and freelancer fulfill their mutual contract obligations (the freelancer delivers the work, and the client approves it).

CRITICAL INTERVIEW NOTE:
In your current codebase, there is NO dynamic payment code or smart contract database. The payment safety and escrow features described on the homepage are visual mockups (design-only prototypes). The functional parts of the project are user authentication, Firestore database integration, dynamic job search/filtering, and the contact form email dispatcher.

How to answer when the interviewer asks: "How does the Escrow work in your code?"

Do not try to make up a fake payment system. An experienced developer will immediately check the code files and catch the mistake. Instead, answer professionally like an architect:

"Currently, the Escrow Payment System is represented as a frontend prototype. It outlines the client-freelancer

user interface flow. In this version, I focused on user onboarding, Firebase Authentication, Cloud Firestore profile management, and dynamic client-side job filtering.

To make it a production-ready escrow platform, we would integrate a payment gateway API like Stripe Escrow or use Web3 smart contracts (Ethereum/Solidity) to programmatically lock funds in an escrow vault and release them upon milestone approval. This project establishes the secure user dashboard and job search foundation required for that integration."

5. Comprehensive Interview Q&A Prep

Prepare these questions and answers to show complete confidence and clarity during your project presentation.

Q1: Can you give a brief overview of your project?

Answer: "This project is a web-based Freelance platform prototype with an integrated Escrow theme. It allows freelancers to view job listings, search for relevant opportunities, and filter them by job type (Full Time, Part Time, Freelancer). It features a fully secure authentication flow using Google Firebase, where users can register (writing profile details to Cloud Firestore) and sign in. It also contains a functional contact form that uses PHP on the backend to email support queries."

Q2: What is the tech stack of your project?

Answer: "For the frontend structure, I used HTML5. Styling is done entirely with Vanilla CSS3. The client-side interactivity (such as job searching, type filtering, and navbar toggle) is written in native JavaScript (ES6+). For user management and database storage, I integrated Google Firebase (Firebase Auth and Cloud Firestore). On the backend, I used PHP for the server-side email dispatch script."

Q3: Why did you choose Firebase instead of standard databases like MySQL?

Answer: "Firebase is a Backend-as-a-Service (BaaS) that significantly simplifies modern web development. It handles secure encryption, user session storage, and database management out of the box. Using Cloud Firestore (a NoSQL database) allows us to save user profiles dynamically as JSON-like documents. This eliminates the need to host and maintain a separate backend database server like MySQL and write manual SQL queries, saving development time and improving app loading speed."

Q4: How does user authentication work in this application?

Answer: "When a user signs up, their data is sent to Firebase Auth using the createUserWithEmailAndPassword() function. When they sign in, we run signInWithEmailAndPassword(). Once authenticated, Firebase returns a unique User UID. We store this UID in the browser's localStorage. On the homepage, Logout.js checks for this UID, fetches the matching profile document from Firestore, and displays the user's name. When they click logout, we delete the UID from localStorage, ending the session."

Q5: What is the purpose of localStorage in your project?

Answer: "LocalStorage is a built-in web browser storage object. It allows us to persist user sessions. When the user logs in, we save their unique UID in localStorage. This prevents the user from being logged out when they refresh the page or close the tab. When they click logout, we clear this storage using localStorage.removeItem() to end the session."

Q6: How does the job search bar work?

Answer: "The search bar operates on the client-side using jobs/jobSearch.js. It listens for input events. Whenever the user types a letter, the script grabs the input value, loops through all job elements in the jobs-container, and reads their <h3> tags containing the job titles. It uses Javascript's .includes() method to check if the typed letters are present in the title. If yes, it displays the card; if not, it hides it by setting display to 'none'."

Q7: How does the job filtering category system work?

Answer: "The job filtering works using jobs/job-list.js. The HTML job cards have a custom attribute called 'data-item' (e.g. freelancer, fullTime, partTime), and the filter tabs have a 'data-target' attribute. When a user clicks a tab, JavaScript reads its target. It then loops through all job cards. If a card's data-item does not match the target (and the target is not 'all'), the script adds a CSS class called '.delete' to the card. In css styles.css, the '.delete' class is styled with 'display: none', hiding the unmatched jobs instantly."

Q8: What does the PHP file do, and how is it connected?

Answer: "The contact page form (contact.html) has its action attribute set to send_email.php. When a user submits the form, it sends a POST request with the form fields. The send_email.php script extracts the name, email, and message. It then uses PHP's built-in mail() function to send a formatted plain-text email to the administrator's address. It returns HTTP code 200 on success, or 500 if the mail server fails."

Q9: Why does your PHP script run on the server side while your JavaScript runs on the client sic

Answer: "JavaScript runs in the user's web browser (client-side) to provide instant, smooth UI changes, like filtering jobs without reloading the page. However, sending emails directly from the browser is insecure and difficult because it requires exposing email credentials or server keys. PHP runs on the hosting server (server-side). This keeps mail server configurations secure from the public, processing the request safely behind the scenes."

Q10: What is NoSQL, and how does Firestore organize data?

Answer: "NoSQL stands for 'Not Only SQL'. Unlike traditional relational databases (like MySQL) that store data in rigid tables with fixed rows and columns, NoSQL databases like Firestore store data in flexible documents. Firestore organizes data into Collections (which are lists of records) and Documents (individual records containing key-value pairs). In our project, we have a 'users' collection, where each document has the user's unique UID as its filename, storing keys like firstName, lastName, and email."

Q11: What is event.preventDefault() and why did you use it in your auth scripts?

Answer: "By default, HTML forms submit data to a backend server, causing the entire webpage to reload. In firebaseauth.js and firebaseSignin.js, we call event.preventDefault() inside the form submission listener. This blocks the browser's default page-reload behavior. This allows us to handle the registration and login asynchronously in JavaScript, displaying success or error messages smoothly without disturbing the user's interface."

Q12: If you could add more features to this project, what would they be?

Answer: "To take this project from a prototype to a production platform, I would:
1. Integrate Stripe Escrow API or Smart Contracts to enable secure payment hold and release.

2. Create dual dashboards: an Employer dashboard to post jobs, and a Freelancer dashboard to apply and submit work.
3. Make job listings dynamic by storing them in a Cloud Firestore 'jobs' collection instead of hardcoding them in HTML. This would let employers add new jobs in real-time."

Q13: How does responsiveness work in your CSS?

Answer: "I used CSS Media Queries to make the website responsive across mobile, tablet, and desktop viewports. Using breakpoints (like @media screen and (max-width: 768px)), the layout changes. For example, the navigation list changes from a horizontal bar into a vertical sliding menu, elements stack vertically instead of sitting side-by-side, and image/font sizes scale down to fit small screens."

Q14: How does Firebase handle password security?

Answer: "One of the major benefits of using Firebase Authentication is security. When a user registers a password, it is not saved as plain text. Firebase securely hashes and salts the password on their servers before storing it. Even if our database database is compromised, passwords cannot be read. Firebase also protects against common attacks like brute-force logins automatically."

Q15: How does the toggle.js script make the mobile navigation menu work?

Answer: "On small screens, we hide the main navigation links behind a hamburger icon (with the ID 'bar'). The toggle.js script listens for a click on this icon. When clicked, it toggles a class called 'active' on the navigation menu list. In CSS, the menu is styled to slide into the viewport when the 'active' class is present, providing a smooth mobile navigation experience."